

Analysing Sigmoidal Models using Sicegar

Federica Domecq Lacroze

2024-01-01

1 The Context: Evaluating the Reliability of Sicegar for Timing

Analysis of RpoS-Dependent Gene Expression

Understanding the precise timing of gene transcription is essential for deciphering cellular responses to various environmental stresses. In the recent study by Adams et al. (2023), the authors investigated the transcriptional timing of RpoS-dependent genes in *Escherichia coli* across different stress conditions, revealing significant variability in transcriptional responses.

To analyse the transcriptional timing of the bacteria under different stresses, the authors of this study recorded RNA-seq data collected at various time points following the induction of stress. The data included the expression levels of numerous genes over time recorded at discrete time intervals, which allowed for the temporal analysis of transcriptional responses. These samples were then passed through an R package, `sicegar`, which is used to model single-cell growth. The gene expression patterns were categorized into distinct models, and their onset times were estimated—

the specific points in time at which gene expression begins to significantly increase, marking the initiation of the gene’s response to the environmental condition or stress being studied.

The current research aims to validate or question the effectiveness of **sicegar** in correctly analysing sigmoidal and double-sigmoidal curves from experimental data, offering recommendations to its users. The tool is not only useful to scientists looking to analyse such transcriptions, but is also particularly useful in any biological study where these types of shapes are prevalent, as in growth curves and dose-response curves.

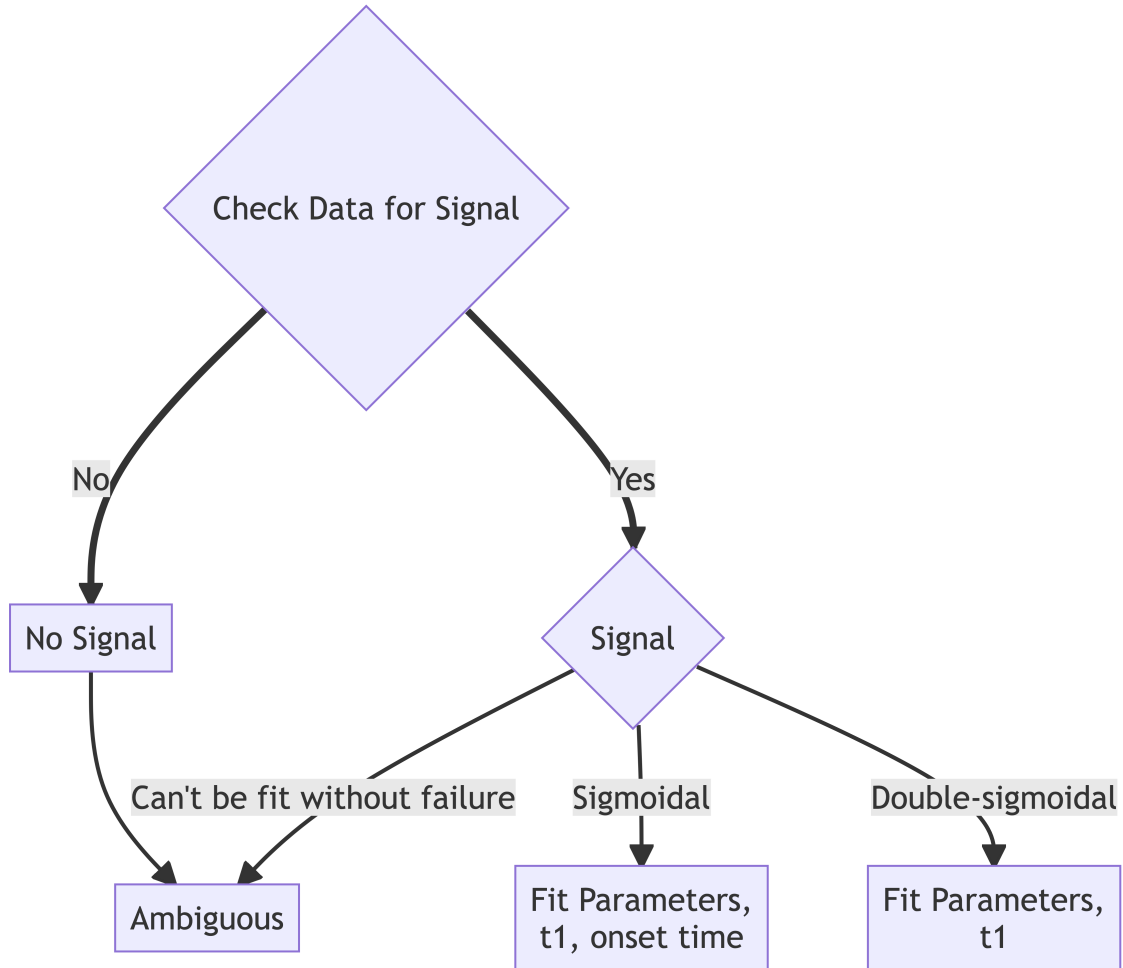
Our simulation based approach aims to assess whether **sicegar** can reliably detect sigmoidal and double-sigmoidal curves and accurately estimate onset times. This validation is crucial since accurate modeling tools are essential for correctly interpreting gene expression dynamics. Ensuring that **sicegar** performs well under various conditions and testing its limits will provide confidence in its application to biological research, supporting robust and reliable conclusions in research focused on cellular responses and gene regulation.

2 More on **sicegar**’s approach

The automated fitting and classification tool, **sicegar**, (Caglar, Teufel, and Wilke 2018) takes in data which seemingly presents either a sigmoidal or double-sigmoidal pattern. It first checks the data for the presence of signal, that is, it checks whether the intensity curve deviates significantly from a horizontal line and varies significantly. It then proceeds to fit the two mathematical models

to the data, automatically determining which curve provides the better fit, by reporting a range of parameters about the fitting models that enables the algorithm to compare the fitting of these curves.

It seems significant to note, that before fitting and categorising the data, it is internally normalised to the domain $[0, 1]$ for both the x and y axis. The models are fitted internally via a likelihood maximisation method, implemented through the function `nls.lm` found in `minpack.lm` package (Elzhov et al. 2016), which performs a modified Levenberg algorithm to minimise the sum of squared residuals (More 1978). For each fitted model, the algorithm returns an AIC score, and we are returned the best fit across replicates after the likelihood maximisation function is run multiple times on our data and multiple models are observed.



2.1 A little background on our favourite curves

As hinted earlier, the expression patterns that are identified and recovered by this software are two: sigmoidal and double-sigmoidal data.

2.1.1 Sigmoidal Data

Sigmoidal models describe gene expressions that exhibit a singular biological phase: intensity increases exponentially until its maximum saturation is reached, at which point it levels off at an asymptotic level. The singular continuous increase or decrease in response to a stimulus, results in S-shaped curves that can be characterised by the following formula:

$$f(x) = h_0 + \frac{h_1 - h_0}{1 + e^{-a(x-t_1)}}$$

. Where the variables correspond to:

- h_0 : the initial or baseline value of the function as x approaches negative infinity. It's the lower asymptote of the sigmoid curve, and for the purpose of our research will always be set to 0.
- h_1 : the final value of the function as x approaches positive infinity. It is the upper asymptote of the sigmoid curve.
- a : the steepness or growth rate of the sigmoid curve. A larger value of a makes the transition from h_0 to h_1 steeper, whilst a smaller value does the opposite.
- t : the midpoint of the sigmoid curve, which we will later on argue is the inflection point and therefore the onset time. It is the value of x at which the function value is halfway between h_0 and h_1 .

2.1.2 Double-Sigmoidal Data

In contrast, double-sigmoidal models are employed when, once the intensity has reached its maximum, it once again decays to a lower one or even back to 0 (which may or may not be observed depending on the system, length of observations and whether the second phase reaches said value).

These two distinct phases of change come together in more complex curves with two S-shapes.

By taking the product of two simple sigmoidal structures, we can end up with a formula that best defines our double-sigmoid:

$$f(x) = \frac{1}{h_1} \left(h_0 \frac{h_1 - h_0}{1 + e^{-a_1(x-t_1)}} \right) \left(\frac{h_1 - h_2}{1 + e^{-a_2(x-t_2)}} \right)$$

Where the variables correspond to:

- h_0 : initial or baseline value as x approaches negative infinity (lower asymptote of the first sigmoid)
- h_1 : scaling factor and common upper asymptote for the first sigmoid and lower asymptote for the second sigmoid
- h_2 : final value as x approaches positive infinity (upper asymptote of the second sigmoid)
- a_1 : steepness or growth rate of the first sigmoid curve
- a_2 : steepness or growth rate of the second sigmoid curve
- t_1 : midpoint of the first sigmoid curve, which we will argue is not the inflection point
- t_2 : midpoint of the second sigmoid curve

The developers of `sicegar`, Caglar, Teufel, and Wilke (2018), argue highlight that the software fails gradually with increasing noise levels and that it conservatively assesses whether it correctly realises that data is sigmoidal or double-sigmoidal (it will return ambiguous rather than hint otherwise).

2.2 The First Roadblock

The main concern of this study is to assess whether this software correctly estimates the onset time of gene transcription, that is, the first inflection point of our sigmoidal and double-sigmoidal curves. This research is built on a simulation-based approach: starting from a place of omniscient truth, we feed the software limited and noisy data and calculate the error of the software’s estimate by comparing it to the underlying signal the data was originally built from.

It’s important to note that the knowledge of the formal definition of these curves does not fully encapsulate the breadth of knowledge we require to test the software in the way we need. Before making any sort of analysis we need to know what the true inflection or onset times are of any artificially pre-defined curves with sigmoidal and double sigmoidal trends. The first step of this investigation was to work towards finding the true inflection or onset time underlying this type of data.

As a first step, traditional mathematics equipped with the formal definitions of these sigmoidal curves, naturally guided us to employ calculus techniques in order to analytically derive the inflection points of these functions.

2.2.1 Sigmoid's Onset time

For the single-sigmoidal shape, it was initially believed that t_1 , the midpoint of the curve, was this onset time. Using symbolic calculus, we showed this to be true:

We first find the second derivative of our function:

$$f(x) = h_0 + \left(\frac{h_1 - h_0}{1 + e^{-a(x-t_1)}} \right) = h_0 + (h_1 - h_0) (1 + e^{-a(x-t_1)})^{-1}$$

$$f'(x) = (h_1 - h_0)(-1)(1 + e^{-a(x-t_1)})^{-2} \cdot (-a) \cdot e^{-a(x-t_1)} = a(h_1 - h_0)(1 + e^{-a(x-t_1)})^{-2} \cdot e^{-a(x-t_1)}$$

$$\begin{aligned} f''(x) &= a(h_1 - h_0) [(-2)(1 + e^{-a(x-t_1)})^{-3} \cdot e^{-a(x-t_1)} \cdot (-a) \cdot e^{-a(x-t_1)} + (1 + e^{-a(x-t_1)})^{-2} \cdot e^{-a(x-t_1)} \cdot (-a)] \\ &= a(h_1 - h_0) [(1 + e^{-a(x-t_1)})^{-2} \cdot (-a) \cdot e^{-a(x-t_1)} \cdot (-2)(1 + e^{-a(x-t_1)})^{-1} \cdot e^{-a(x-t_1)} + 1] \end{aligned}$$

We equate our second derivative to 0 and then solve for the inflection point:

$$2 \cdot (1 + e^{-a(x-t_1)})^{-1} \cdot e^{-a(x-t_1)} = 1 \Rightarrow e^{-a(x-t_1)} = \frac{1}{2}(1 + e^{-a(x-t_1)})$$

$$\ln \left(\frac{1}{2}(1 + e^{-a(x-t_1)}) \right) = \ln \frac{1}{2} \Rightarrow -a(x - t_1) = 0$$

$\Rightarrow x = t_1$ is the point of inflection point and it is independent of our value of h_0

2.2.2 Double-Sigmoid's Onset Time

Unlike the single sigmoid curve. The inflection point of this curve was not as easily defined. Even when we set the slope of a_1 and a_2 to be equal, simplifying the function of a “symmetric” double sigmoid $f(x) = \frac{1}{(1+e^{-a(x-t_1)})} \cdot \frac{1}{(1+e^{a(x-t_2)})}$, it was not evident that we could simply take a double derivative and solve for x when the function was equal to 0. In fact, when we did we were left with whatever the solution to this is:

$$e^{-2t}y^4 - (e^{-t} - e^{-2t})y^3 - 2e^{-t}y^2 - (1 + e^{-t})y + 1 \text{ where } y = e^x$$

Not trivial at all. We were able to conclude however, through some experimentation with graphing tools such as Desmos, that even when the slopes on either side of our double sigmoid coincided, the onset time was not exactly t_1 - in fact this value at times is significantly off.

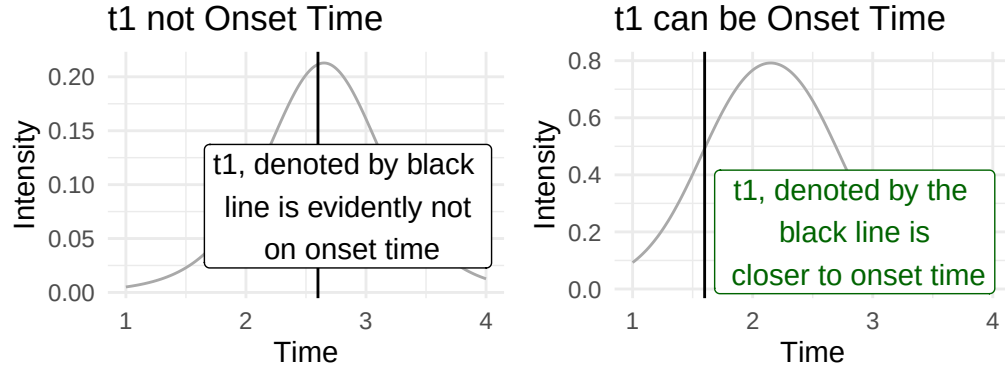


Figure 1: $a = -3.1$, $t_1 = 2.6$, $t_2 = 2.7$ for the first graph and $a = 3.8$, $t_1 = 1.6$, $t_2 = 2.7$ for the second graph for a simplified model of a double sigmoid where $a = a_1 = -a_2$. From observation, it seems as though how close t_1 is to onset time is dependant on both these slope values and the proximity of t_1 and t_2 themselves.

For the purpose of our simulation-focused approach, we needed an easy way to derive the first inflection point of any double-sigmoidal curve we could be interested in. We tackled the issue in a more computational and systematic way.

Our simulations allowed us to choose the values of all our variables; instead of having a singular cumbersome solution for the double derivative of our formula, we realised that we could choose our variables and then use computational numerical approximation methods to calculate the inflection point of our newly simplified double sigmoids. The steps we defined were as follows:

1. Define our parameters
2. Redefine our function by substituting these parameters in
3. Calculate the second derivative of this simplified function
4. Solve for the roots

Example

1. Defining the parameters corresponding to the double sigmoid we want to look at:

```
dblsig_h0 <- 0
dblsig_h1 <- 700
dblsig_h2 <- 300
dblsig_t1 <- 230
dblsig_t2 <- 250
```

```
dblsig_a1 <- 0.05
```

```
dblsig_a2 <- 0.01
```

2. Redefining our function by substituting these parameters in, and storing this definition in R:

```
dbl_sig_func <- function(x, h0, h1, h2, t1, t2, a1, a2) {  
  (1 / h1) * (h0 + (h1 - h0) / (1 + exp(-a1 * (x - t1)))) * (h2 + (h1 - h2) /  
  (1 + exp(a2 * (x - t2))))}
```

3. Using the package Deriv to calculate the double derivative of our pre-defined function:

```
dbl_sig_pri <- Deriv::Deriv(dbl_sig_func, "x")  
dbl_sig_pri_pri <- Deriv::Deriv(dbl_sig_pri, "x")  
dbl_deriv_specif <- function(x) {  
  dbl_sig_pri_pri(x, h0 = dblsig_h0, h1 = dblsig_h1, h2 = dblsig_h2,  
    t1 = dblsig_t1, t2 = dblsig_t2, a1 = dblsig_a1, a2 = dblsig_a2)}
```

4. Solving the second derivative for the roots of the simplified function using Uniroot:

```
true_inflec <- uniroot(dbl_deriv_specif, c(0, mean(c(dblsig_t1, dblsig_t2))))$root
```

* Note that `uniroot` requires a window in which the inflection point of interest resides. With certainty, if we start recording at $t = 0$, we can say that this inflection point is between then and the mean of our t values.

3 Simulating

Once we knew the key ingredients of the underlying models that we were interested in, we proceeded to simulate data to test the accuracy of this equipment. In this context, simulation involves generating synthetic data that mimics real experimental conditions of gene transcription under stress. We generate the synthetic data using our known and understood mathematical models with controlled parameters. By introducing various levels of noise and different testing conditions, researchers can create a wide range of scenarios to test the performance of analytical tools.

The primary purpose of using simulation here is to validate the effectiveness of the **sicegar** package. When we carry out scientific experiments, we never fully know the real “truth”, in fact: make a hypothesis; collect data; and then use technology to test this hypothesis, trying to get closer to understanding the underlying patterns and truths. During a simulation, we manipulate data whose origin we know and understand (because we are the ones generating it). By feeding the tool synthetic data with known characteristics, we can assess whether the technology accurately detects sigmoidal and double-sigmoidal patterns and estimates onset times correctly. It’s like writing an exam and a markscheme to test an individual’s understanding of known matter. Simulating provides a controlled environment where the exact parameters and true values are known, allowing for precise measurement of the tool’s accuracy and reliability. Similarly generating data under varying conditions with different levels of noise allows us to better understand how well a tool performs under ideal conditions versus more challenging real-world scenarios.

3.1 Generating Artificial Data

The first part of running these simulations was generating data that not only resembled that of experimental conditions and limitations, but whose backbone we fully understood. To do such a thing, we did the following:

- Defined the model we wanted to create data of (as defined in section 2.1):
 - Sigmoidal
 - Double-sigmoidal
 - Other
- Generated these strictly defined curves by choosing the parameters that defined the model of interest and “plotting” the curves; computed the true value of inflection as described in section 2.2
- Made “sampling decisions” to make data resemble naturally generated data (removing observations or make the data more noisy):
 - **x-limit**: how long we record data for, the time of our last observation
 - **dispersion**: measure of variance, how much our observations can vary from the underlying model
 - **n-reps**: number of observations at each time point
 - **by-x**: time between observations
- Removed the underlying model, and then passed our data into the software

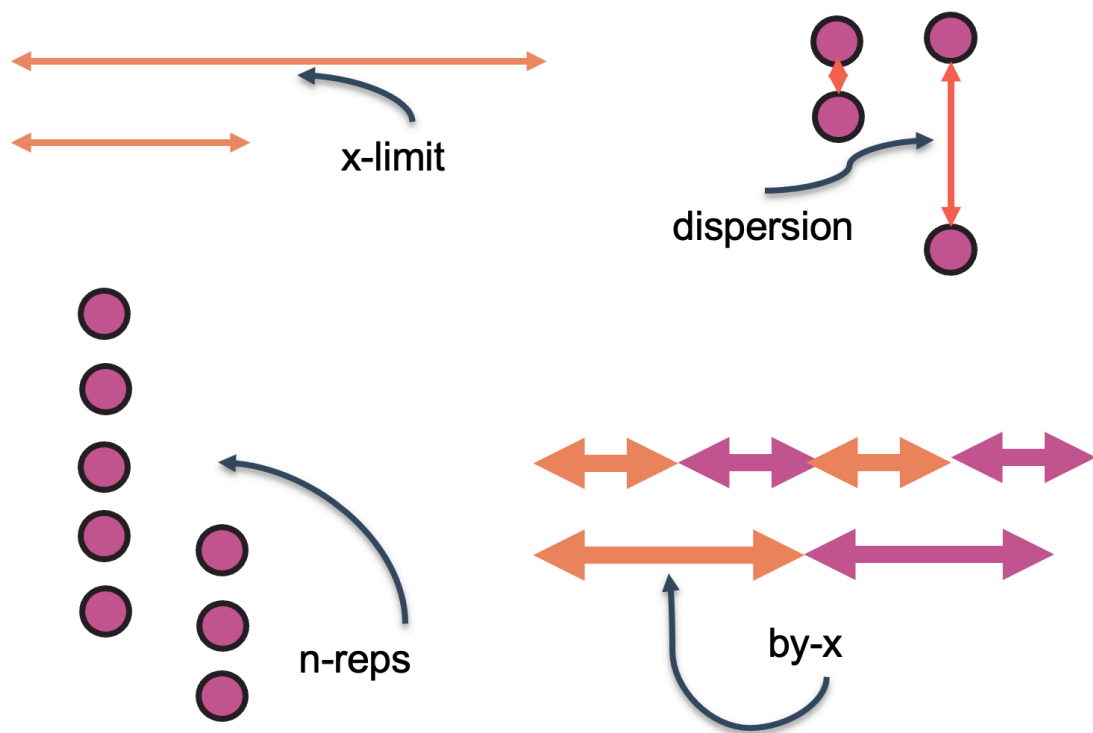


Figure 2: The “sampling decisions” we have control over as experimenters

3.1.1 Example of Generating Data:

Sigmoid Model

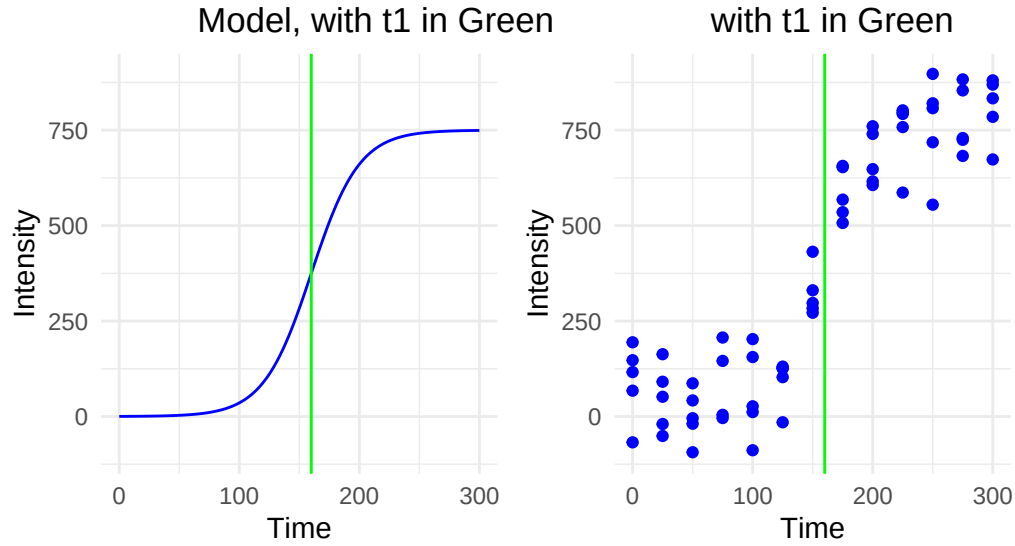
```
sig_func <- function(x, h0, h1, t1, a){  
  h0 + (h1 - h0) * (1 / (1 + exp(-a * (x - t1))))}
```

Choosing Parameters, calculating inflection point t_1

```
sig_h0 <- 0  
sig_h1 <- 750  
sig_t1 <- 160  
sig_a <- 0.05
```

*Note, in the original source code, we shifted all of our observations four times the dispersion up - a common error we encountered when running our models seemed to be due to the range of values which found themselves under the x-axis. This precaution measure while not perfect (alters the nature of our data), provided a fix that would seem inconsequential to the computation of the onset time which should be independent of this shift.

Example of Underlying Sigmoid Example of Generated Data



3.2 Feeding Data into `sicegar`

Before looking at what investigations we performed and discussing our findings, it's important to highlight that when using `sicegar`'s `fitAndCategorize`, we altered some of the default modelling parameters to match those used in the study (Adams et al. 2023) that prompted this research. We set `threshold_t0_max_int = 1E10`, `threshold_sm_tmax_dataRatio = 0.65`, `threshold_AIC = 5`. As described by documentation (Caglar, Teufel, and Wilke 2018), this is what the following do:

- `threshold_t0_max_int`: maximum allowed intensity of the fitted curve at time is equal to zero ($t=0$). (Default is 0.05, and the values are based on actual, not the rescaled data.).

- `threshold_sm_tmax_IntensityRatio`: threshold for the minimum intensity ratio between the last observed time points intensity and theoretical maximum intensity of the sigmoidal curve. If the value is below the threshold, then the data can not be represented with the sigmoidal model. (Default is 0.85).
- `threshold_AIC`: maximum AIC values in order to have a meaningful fit (Default is -10).

4 Simulating and Analysing

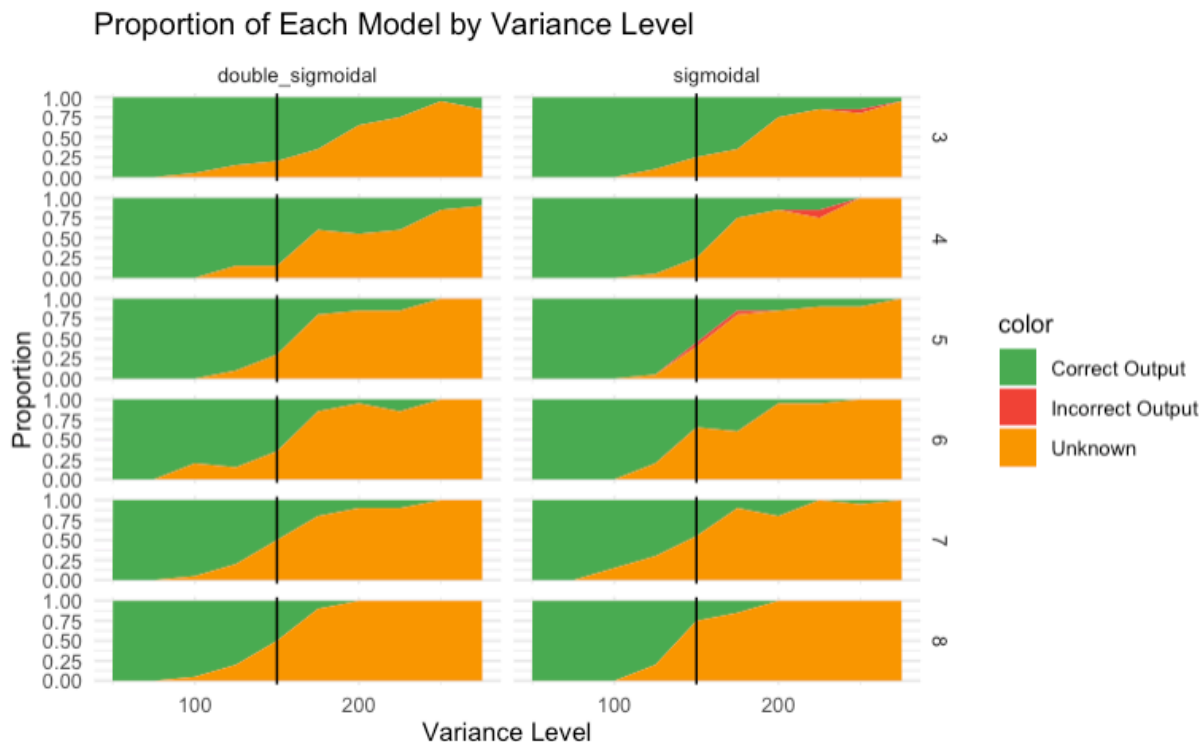
4.1 A Sanity Check

Before running simulations with the purpose of determining how good of an onset time estimator `sicegar` really is, we first asked ourselves a simpler question: is the software determining the model correctly? How fast do we break the technology... that is, how varied does the data have to be until the software runs towards categorising all models as ambiguous? We ran generated models based off of singular sigmoid and double sigmoid whose parameters seemed to resemble that of Adams et al. (2023), and played with both dispersion (`disp`) and number of observations at each time point (`n_reps`).

Sig.	h0	h1	t1	a	D-Sig.	h0	h1	h2	t1	t2	a1	a2
	0	750	160	0.05		0	750	300	160	250	0.05	0.1

“Sampling Decisions”

x-lim	by_x	dispersions	n_reps	reps
300	25	seq(50, 275, by = 25)	3:8	20



From our small sample size we were able to observe that, indeed the software conservatively assesses whether it correctly realises that the data is sigmoidal or double-sigmoidal. In very few instances did we observe the software misidentifying the models. Interestingly, we only saw a few instances of sigmoidal models with high variability being categorised as double-sigmoidal and none in the other direction.

While the number of observations at each time point did not seem significant in changing the rate at which these models were identified (3 versus 8 observations at each timepoint basically returned the same output and trend), at a certain and similar point for both sigmoidal and double sigmoidal curves we observed a spike in the proportion of models categorised as ambiguous. We were encouraged by the lack of incorrectly categorised models and therefore proceeded to continue with our analysis.

4.2 Finally Simulating

We proceeded to run simulations on both sigmoid and double sigmoid models, jotting down all of our observations. We especially played with `n_reps`, `disp`, `x_lim` and selected parameters that resembled the data collected in Adams et al. (2023) for the shapes of our sigmoid and double sigmoid models.

4.2.1 Outcome of Single Sigmoid

The most notable observation is that as dispersion increases, so does `sicegar`'s error. As can be observed in the following plot, where the width of the bar is indicative of the dispersion of the data from its underlying model.

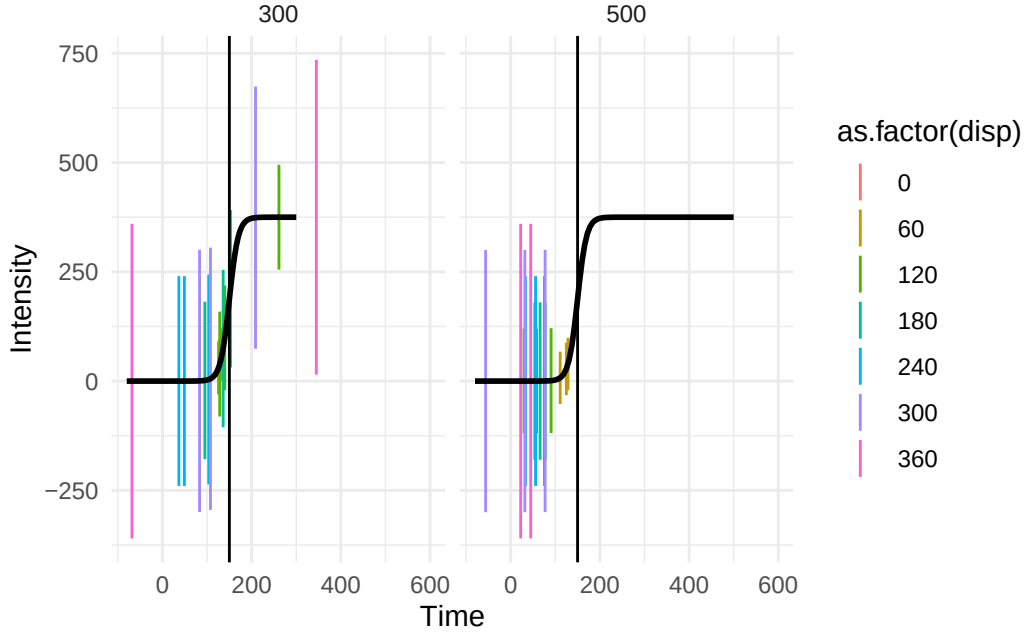


Figure 3: an example of the returned onset times by `sicegar` when we choose $h_0 = 0$, $h_1 = 375$, $a = 0.1$, $t_1 = 150$, $n\text{-reps} = 5$, and $x\text{-lim} = 300, 500$

Very evidently, the data seems to more consistently underestimate the onset time of the curve. However it was notable that if onset time was too close (in this case for example when the x-limit was set to 300), the software was also capable of greatly overestimated this time. When the x-limit was high enough, we could anticipate `sicegar` to underestimate the model; the higher the dispersion, the more the estimate was pushed out in whichever direction we generally saw the miss. Seemingly, the distance between the onset time, t_1 , and the time of the last observation recorded, $x\text{-lim}$, mattered.

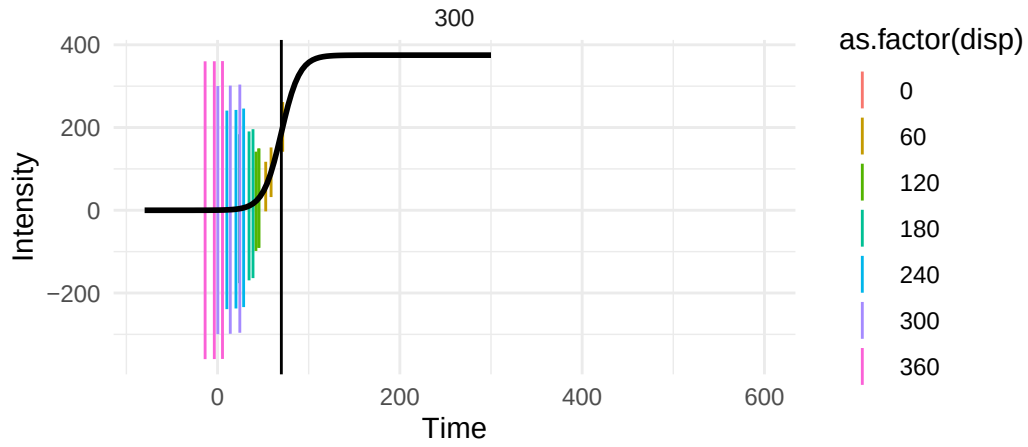


Figure 4: an example of the returned onset times by sicegar when we choose $h_0 = 0$, $h_1 = 375$, $a = 0.1$, $t_1 = 70$, $n\text{-reps} = 5$, and $x\text{-lim} = 300$

It was also valuable to note that output variation was a lot higher when the model was noted to be ambiguous. As observed in the following plots:

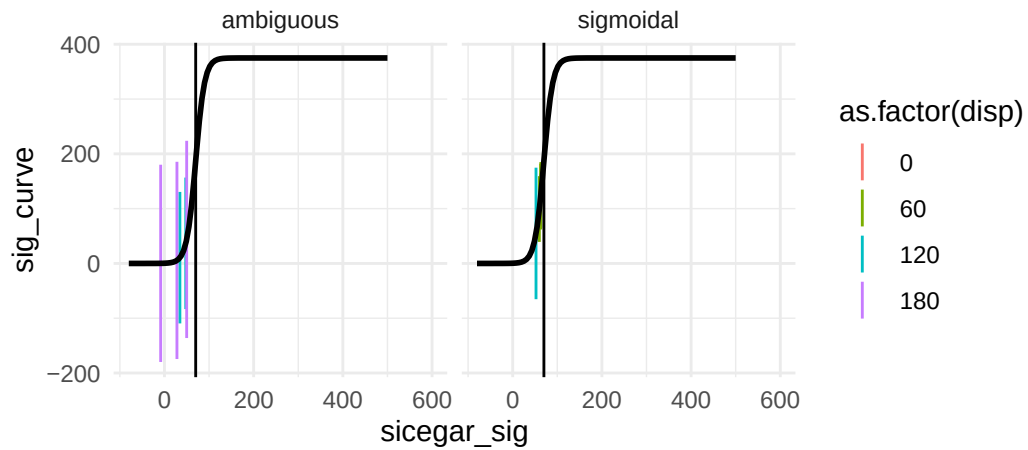


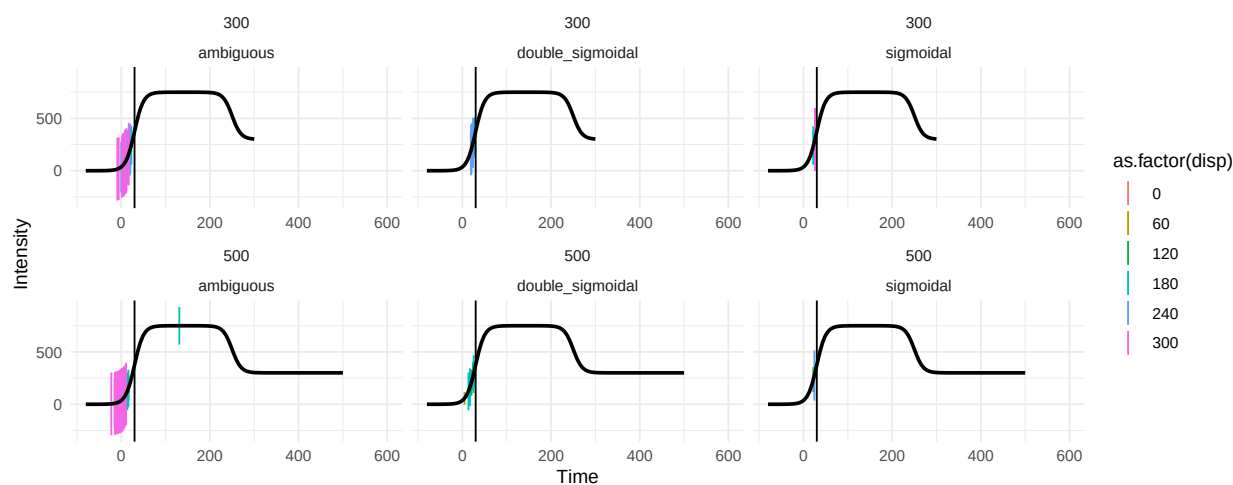
Figure 5: an example of the returned onset times by sicegar when we choose $h_0 = 0$, $h_1 = 375$, $a = 0.1$, $t_1 = 70$, $n\text{-reps} = 5$, and $x\text{-lim} = 500$, for different model outputs

An encouraging finding, however, was that when dispersion was 0, the software detected $\mathfrak{t}1$ accurately 100 % of the time.

4.2.2 Outcome of Double Sigmoid

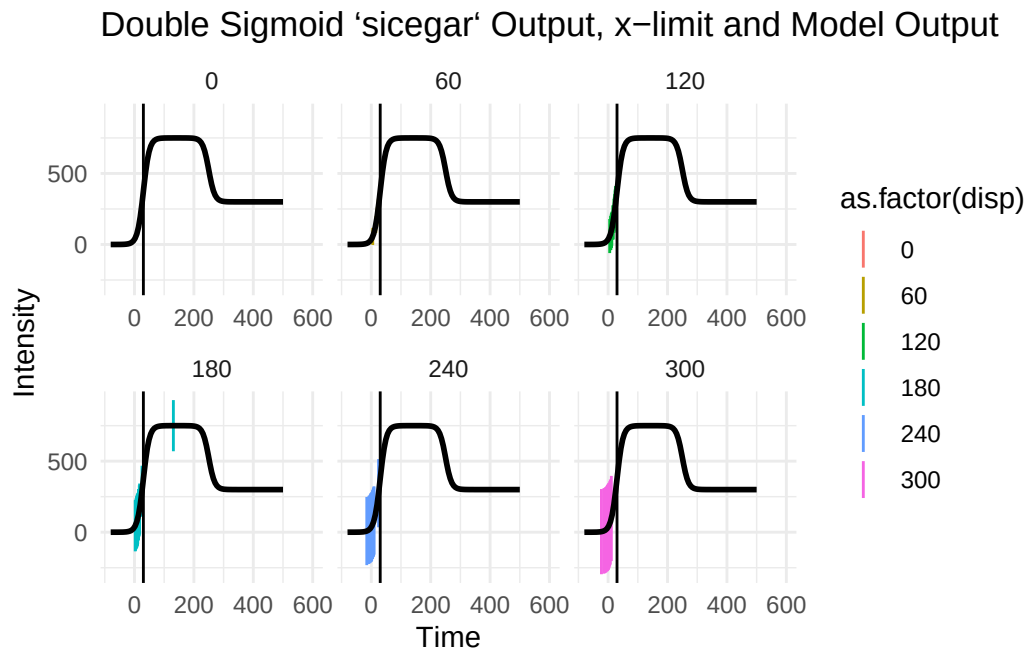
It's important to note that as shown earlier, the first point of inflection of the double-sigmoid, more often than not, is not $\mathfrak{t}1$. The value that `sicegar` returns is supposedly $\mathfrak{t}1$, however. We must not mistakenly take this value to be the true onset time in every instance.

It is also valuable to note that the software seems to return values and optimise these models faster when double-sigmoids are being run through.



As we can observe above, regardless of whether the model is correctly identified, like our single-sigmoid data, the software tends to underestimate our onset time value. The software follows a similar pattern to the single sigmoid - as dispersion increases, the window of return values shifts

further away in the direction of the error. We do note, however, that unlike the single sigmoid, there doesn't seem to be an evident relationship regarding x-limit and the accuracy of `sicegar`'s estimate.



5 Bootstrapping

Having observed the consistent underestimation of onset time for both of our models, we set out to look for a fix or a recommendation to biologists. We decided to use the method of bootstrapping (Efron and Tibshirani 1993) to calculate the amount of bias the software presented, attempting to provide a fix that got us closer to the truth.

Bootstrapping is a powerful statistical technique that can be useful for estimating and correcting

bias in estimations. By resampling the original dataset with replacement, bootstrapping generates multiple simulated samples, each of which is used to calculate the statistic of interest. This process creates an empirical distribution of the statistic, which allows for the assessment of bias. Bias, defined as the systematic deviation of the estimated value from the true value, can be identified by comparing the mean of the bootstrap estimates to the original estimate. Once detected, this bias can be adjusted to improve the accuracy of the estimate.

We were able to complete the computation and method to perform a bootstrap for sigmoidal models.

5.1 Parametric Bootstrapping for Sigmoidal Models

Despite making minimal assumptions about the form of the population distribution, to perform a successful bootstrap we assume that the sample is representative of the population and that the resampling of this data can closely resemble the data that we worked with. Our data at first presented a couple of difficulties. Our data was sampled at discrete time points with a few observations at each, and the size of each simulated sample was small. To perform a good exercise, we had to ensure that we selected (like in the original data) 5 observations at each time point - not any set of data points could be randomly selected from all. Similarly, this presented another issue, having only 5 observations at each time point to sample from and performing regular bootstrapping from each set of 5, we would likely resample the same values over and over to generate a model, overlaying them on top of each other and straying from the original model design. To work with

these issues, we decided to perform a parametric bootstrap. Where we estimated the parameters of the models we generated data from (calculated dispersion etc) and created bootstraps with the same method of data generation we originally had. We performed the following:

1. Set the parameters, created the function, and simulated the original dataset to perform a bootstrap on as per usual.
2. Ran `sicegar` for this original data set and saved the values computed, especially `onset time`.
3. Using the model from the data, found a new set of parameters (that is, parameters estimated by `sicegar` on the original data).
4. Using the `sicegar` parameters, generated a single bootstrapped dataset with our original data generation method, just changed to estimated parameters.
5. Applied `sicegar` again and found the bootstrapped value of our onset time.
6. Created 100 bootstraps on 500 datasets and then calculated the difference between the original onset value calculated and the second one calculated on the bootstrap.
7. Adjusted the original onset time calculated by adding back the mean error that the bootstrap calculated.

We were able to observe what we expected. The original sample data set onset times, despite being less variable in range of values, more consistently underestimated the true onset time, and the bootstrap bias fix got us a lot closer to the real onset time value. Our bootstrap generated estimates were centred a lot closer to the real value.

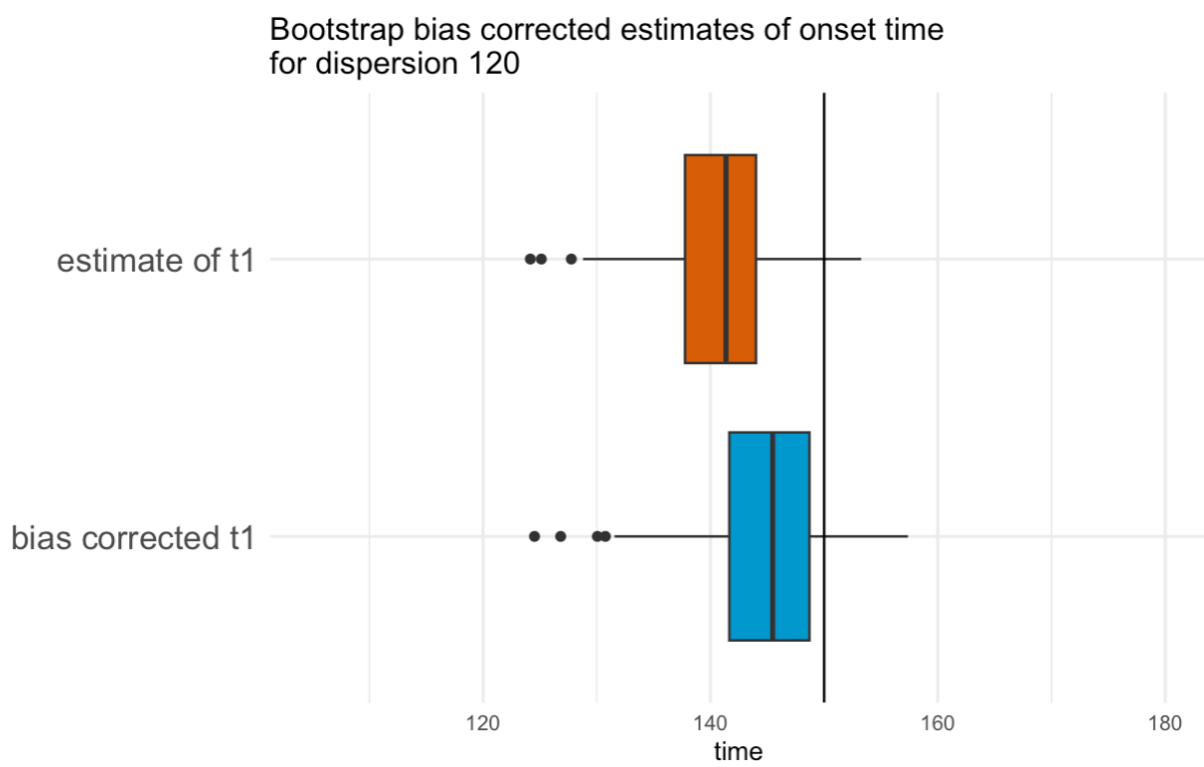


Figure 6: Boxplot of onset time estimate improvement with bootstrap

5.2 Parametric Bootstrapping for Double Sigmoidal Models

There is no reason to believe that the same method would not help us breach the underestimation for the double sigmoidal model, however we have not yet been able to complete the code to run this type of simulation at a large scale. The computation would be more complicated and demanding because the bootstrap would require us to run parallel simulations within parallel simulations. The double sigmoid's onset time is not `t1` and therefore we would have to find the inflection point for each of the bootstraps for each of the data sets. Similarly, it would take thinking to determine which parameter outputted from `fitAndCategorize` would correspond to each of the variables in the definition of the double-sigmoid which takes in a lot more variables. An outline of what creating the code for this parametric bootstrapping would look like is:

1. Choose the parameters for the original data model we want to bootstrap on from matching data trends we expect to see in gene expression studies. Call this `OM` (original model)
2. Calculate the inflection point of `OM`, `inflec_om`
3. Generate a model sample from `OM`, we could call it `OMs`
4. Use `fitAndCategorize` to estimate `OMs` onset time and look at parameters for bootstrap sample
5. Once you have the parameters for the “bootstrap model”, `bsM`, calculate the inflection point of `bsM_om` from the parameters and name this `bsM_om_trueinflec`
6. Generate sample from `bsM`, we could call it `bsMs`
7. Run `fitAndCategorize` on `bsMs` and record onset time again

For each of the 500 samples of `OM` , create 100 `bsMs` , calculating the onset time for each and comparing to the corresponding `bsM_om_trueinflec`

6 Tackling Issues

When performing this research, we oftentimes, when running simulations at scale, would find our work truncated by issues in `fitAndCategorize`. Of the hundreds of thousands of models with changing definition and variability we passed through the function, a limited number would return errors where the modeling function would be unable to get through a layer of its computation. We attempted to resolve this issue by recreating the seed and model that outputted an error on the HPC, however were unable to trigger the same exact results on our native machines. In all of our large scale parallelly run simulations, we implemented `tryCatch`, an error correcting code function that would record the model parameters and seed that returned an error whenever `fitAndCategorize` was unable to resolve an issue. In such a way, we were able to continue our investigation and save the faulty parameters for a later investigation.

7 The Future

This is very much not the end of this project. There is a lot more to be done, and a lot more to be understood.

We can tackle this question in a magnitude of ways, leading us to different answers.

From a more computational perspective, we could dive deeper into the inner layers that make this function work, and the functions that are used to optimise these model estimations. We want to be able to understand where the bias present comes from, and a computational investigation on `sicegar`'s model generation or `nls.lm` optimisation function could shed light into this matter. It would also be interesting to expand on section 4.1, drawing more concrete conclusions regarding what and at what exact point certain parameters make the software default to ambiguous - what makes the software break?

Mathematically, we could look more into the nature of the double sigmoid, a more formal understanding of what drives the value of onset time and getting further on the understanding of onset time estimation as onset time gets further away from `t1`. It would be valuable to find a way to translate the parameters that `sicegar` returns into an actual value for inflection time.

From a simulation-based perspective it would be valuable to run `sicegar` on different models: linear, logarithmic, exponential etc. It would also be important to further investigate the output of more parameter values and configurations, both the controllable parameters and those that can be selected in experimental design. Too many factors go into defining these functions and a clear order in which to investigate is not evident. Despite having run these simulations with a homoscedastic design, it is important to note that the original paper presents data with evident heteroscedasticity. A valuable change in the design of this research would be to implement this in the data test, and compare with our current results.

Last but not least, we would find value in not only comparing this software to other existing packages

but also running tests to evaluate speed and efficiency - at first hand it seems like `sicegar` is faster at analysing double-sigmoidal models.

We must complete a comprehensive literature review of scientific investigations that would find value in these sorts of analyses and draw out relevant conclusions, getting our recommendations to `sicegar` users.

References

- Adams, J, J Hoang, E Petroni, E Ashby, J Hardin, and DM Stoebel. 2023. “The Timing of Transcription of RpoS-Dependent Genes Varies Across Multiple Stresses in *Escherichia Coli* K-12.” *mSystems*. <https://doi.org/10.1128/msystems.00663-23>.
- Caglar, M. U., A. I. Teufel, and C. O. Wilke. 2018. “Sicegar: R Package for Sigmoidal and Double Sigmoidal Curve Fitting.” *PeerJ* 6: e4251. <https://doi.org/10.7717/peerj.4251>.
- Efron, Bradley, and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman & Hall/CRC.
- Elzhov, T. V., K. M. Mullen, A.-N. Spiess, and B. Bolker. 2016. *Minpack.lm: R Interface to the Levenberg-Marquardt Nonlinear Least-Squares Algorithm Found in MINPACK, Plus Support for Bounds*. <https://CRAN.R-project.org/package=minpack.lm>.
- More, J. J. 1978. “The Levenberg-Marquardt Algorithm: Implementation and Theory.” *Lecture Notes in Mathematics* 630: 105–16. <https://doi.org/10.1007/BFb0067700>.